

TRANSFORMAÇÃO DIGITAL

Catálogo Inteligente — EAD Unifor

Disciplina: Desenvolvimento de Software em Nuvem | Data: maio/2026

Projeto	Catálogo Inteligente — aplicação web de catálogo de produtos com filtros avançados
Repositório	https://github.com/PYEDRO/Catalogo_Nuvem
Stack	React 18 + TypeScript + Vite (frontend) FastAPI + Python 3.11 (backend) Firebase Auth + Firestore Cloud Run GCP Firebase Hosting GitHub Actions CI/CD
ODS	ODS 8 — Trabalho decente e crescimento econômico
Prazo	05/06/2026

Estrutura do Relatório

- **Etapa 1 — Diagnóstico:** mapeamento de telas, auditoria de acessibilidade, análise estática de código, avaliação de usabilidade, métricas de performance, prontidão para lançamento, definição de KPIs e classificação de problemas por severidade.
- **Etapa 2 — Implementação:** correções de acessibilidade (antes x depois), refatorações de código, otimizações de performance, melhorias de usabilidade, pipeline CI/CD aprimorado e plano de monitoramento.
- **Etapa 3 — Avaliação:** análise de KPIs, lições aprendidas e planejamento de iterações futuras.

ETAPA 1 DIAGNÓSTICO

Passos 1–7: mapeamento do projeto, auditoria de acessibilidade, análise estática de código, avaliação de usabilidade, métricas de performance, prontidão para lançamento, KPIs e classificação de problemas.

1.1 Mapeamento de Telas, Funcionalidades e Fluxos

O Catálogo Inteligente é composto por três telas principais e um conjunto de componentes reutilizáveis que implementam os fluxos de autenticação, navegação, filtragem e administração de produtos.

Login.tsx	Autenticação via Firebase Auth (e-mail + senha). Redireciona para Home após login.	Tela
Home.tsx	Lista paginada de produtos com filtros (categoria, preço, estoque) e busca textual. Paginação numérica.	Tela
Admin.tsx	CRUD completo de produtos: criar, editar, excluir. Protegido por role 'admin'.	Tela
Navbar.tsx	Navegação principal: link home, e-mail do usuário, acesso Admin, botão Sair.	Componente
FilterPanel.tsx	Painel lateral com filtros de busca, categoria, faixa de preço e estoque.	Componente
CatalogGrid.tsx	Grid responsivo de ProductCards com estados de loading, erro e vazios.	Componente
ProductCard.tsx	Card de produto com imagem, nome, preço, categoria, tags e botões admin.	Componente
backend /catalog	API REST: GET /products (filtros + paginação), GET /products/{id}, POST, PUT, DELETE.	Sistema
backend /auth	Verificação de token Firebase e retorno de role do usuário autenticado.	Sistema

Fluxos Principais Identificados:

- **Fluxo de Navegação:** Unauthenticated → Login → Home. Admin: Home → /admin (ProtectedRoute).
- **Fluxo de Filtragem:** FilterPanel → URL params (useSearchParams) → useCatalog hook → API fetch → CatalogGrid.
- **Fluxo de CRUD:** Admin form → API POST/PUT/DELETE → Firestore → resposta ProductResponse.
- **Fluxo de Autenticação:** Firebase signIn → onAuthStateChanged → getIdTokenResult (claims.role) → ProtectedRoute.

1.2 Auditoria de Acessibilidade (Passo 2)

A auditoria foi realizada com análise manual do código-fonte contra os critérios WCAG 2.1 (AA) e com inspeção de padrões ARIA. Os problemas foram classificados conforme impacto para usuários de tecnologias assistivas (leitores de tela, navegação por teclado).

<label> sem htmlFor/id — leitores de tela não associam campo ao rótulo	login.tsx	3.1 (A)	ALTO
Ausência de skip-to-content link — teclado percorre a cada página	Navbar.tsx	2.4.1 (A)	ALTO
Botões de paginação sem aria-label — anunciados como '1','2','3' sem contexto	Home.tsx	4.1.2 (A)	ALTO
Inputs do FilterPanel sem id/htmlFor vinculados	FilterPanel.tsx	1.3.1 (A)	MÉDIO
Grid de loading sem role=status/aria-busy — leitores não sabem se carregando	CatalogGrid.tsx	4.1.3 (A)	MÉDIO
Estado de erro sem role=alert — não anunciado	CatalogGrid.tsx / Login.tsx	4.1.3 (AA)	MÉDIO
Nav sem aria-label — múltiplas nav na página causam ambiguidade	Navbar.tsx	4.1.2 (A)	MÉDIO
Emoji decorativo sem aria-hidden — lido como texto por leitores de tela	ProductCard.tsx	1.1.1 (A)	BAIXO
Inputs de preço Mín/Máx sem label visível ou sr-label	FilterPanel.tsx	1.3.1 (A)	BAIXO

1.3 Análise Estática de Código — Dívidas Técnicas e Code Smells

(Passo 3)

A análise estática foi conduzida com revisão manual aplicando princípios Clean Code, SOLID e padrões de arquitetura FastAPI + React. Foram identificados problemas de lógica, performance e manutenibilidade.

config.py usa classe simples sem validação — env vars sem type hints	services retornam "Código não é tipado"	ALTO
query_with_filters() faz query.stream() carregando TODOS os documentos para paginação	Performance — O(n) por req	ALTO
Filtro de busca (search) aplicado APÓS paginação no catalog.py — busca após ordenação	Bug lógico: itens da página atual não são filtrados	ALTO
search não passado para product_service.query_with_filters.py parâmetro id pelo service	Erro de integração	ALTO
Operações Firestore síncronas em funções async — bloqueio de event loop	Performance	ALTO
Violação DRY: lógica de busca duplicada (catalog.py e filters.py)	DRY Violation	MÉDIO
getFiltersFromURL() declarada dentro do componente React	Encapsulamento	MÉDIO
Sem type annotations de retorno nas funções do backend	Manutenibilidade	BAIXO
uploadImage() em api.ts referencia endpoint inexistente	Dead Code	BAIXO

1.4 Avaliação de Usabilidade e Métricas de Performance (Passo 4)

Avaliação Heurística (Nielsen 10 Heurísticas)

H1 — Visibilidade do status	Campo de busca não indica que está em modo de busca	MÉDIO
H3 — Controle do usuário	Exclusão de produto sem confirmação — ação irreversível	ALTO
H6 — Reconhecimento vs. Memória	Paginação sem 'Página X de Y' — usuário não sabe onde está no conjunto	MÉDIO
H8 — Design estético	FilterPanel com sticky top-6 conflitando com Navbar	BAIXO
H9 — Recuperação de erros	Mensagem genérica 'E-mail ou senha inválidos' sem sugestão de ação	BAIXO

Métricas de Performance Coletadas (Baseline)

Métricas coletadas via análise de código e estimativas baseadas na arquitetura atual (sem Firestore de produção disponível localmente):

Métrica	Antes	Meta	Observação
Tempo de resposta GET /products (sem filtros)	~120ms (estimado)	< 200ms	Inaceitável com N>1000 docs
Requests por busca (keystroke)	1 por tecla (~10/palavra)	< 2/palavra	Sem debounce
Imagens carregadas no grid (above-fold)	12 simultâneas	3-4	Sem lazy loading
Cobertura de testes backend	6 testes / ~45%	> 70%	Sem coverage report
Pipeline CI/CD — tempo médio	~4 min (só test+deploy)	< 6 min	Sem lint/type-check

1.5 Prontidão para Lançamento (Passo 5)

Testes automatizados	Parcial (6 testes, ~45% coverage)	Sem coverage report no CI
Pipeline CI/CD	Existe (test + deploy)	Sem lint, type-check ou coverage gate
Estratégia de deploy	Rolling (Cloud Run)	Sem health check pós-deploy

Plano de rollback	Manual (gcloud run services update --no-traffic)	Não documentado no workflow
Monitoramento	Logs apenas (Cloud Logging básico)	Sem alertas, dashboards ou SLOs
Documentação API	Swagger automático (/docs)	Adequado para time interno
Variáveis de ambiente	Via GitHub Secrets	Adequado, sem validação no boot
CORS	allow_methods=[*] allow_headers=[*]	Permissivo demais para produção

1.6 Definição de KPIs de Impacto (Passo 6)

Foram definidos 6 KPIs alinhados ao ODS 8 (Trabalho Decente e Crescimento Econômico), considerando o contexto do Catálogo Inteligente como plataforma de catálogo de produtos para apoio a micro e pequenas empresas.

KPI	Definição	Meta	Ferramenta
Taxa de Adoção	% de visitantes que criam conta e realizam pelo menos uma busca na plataforma	> 30%	Firebase Analytics + Google Analytics
NPS (Net Promoter Score)	Pontuação de recomendação coletada via pesquisa pós-uso (0–10)	> 50	Formulário in-app + Google Forms
Taxa de Erro de API	% de requisições que retornam status 4xx ou 5xx sobre o total	< 1%	Cloud Monitoring / GCP Error Reporting
Retenção D7	% de usuários que retornam à plataforma 7 dias após o primeiro acesso	> 40%	Firebase Analytics (cohort report)
Tempo Médio por Tarefa (TMP)	Tempo médio para encontrar um produto usando filtros — medido via sessão analytics	< 45s	Hotjar / Firebase Analytics events
Taxa de Cobertura de Testes	% de linhas de código cobertas por testes automatizados (backend)	> 70%	pytest-cov + GitHub Actions artifact

1.7 Classificação de Problemas por Severidade e Plano de Ação (Passo 7)

Todos os problemas identificados nas seções 1.2 a 1.5 foram classificados abaixo e priorizados para implementação na Etapa 2:

Severidade	Problema	Área	Ação
	Paginação in-memory (stream()) de todos os docs	Backend	Cursor-based pagination + COUNT aggregation
	Bug: search aplicado após paginação — resultados incorretos	Backend	Mover filtro para antes da paginação no serviço
	search não passado para query_with_filters()	Backend	Corrigir chamada em catalog.py
	Operações Firestore síncronas bloqueiam event loop	Backend	Envolver com asyncio.to_thread()
	config.py sem validação de env vars obrigatórias	Backend	Migrar para Pydantic BaseSettings
	label sem htmlFor/id no formulário de Login	Frontend	Adicionar id/htmlFor + aria-required
	Exclusão de produto sem confirmação	Frontend	Implementar confirmação inline de 2 cliques
	Sem skip-to-content link na Navbar	Frontend	Adicionar skip link: sr-only + focus:not-sr-only

	Botões de paginação sem aria-label descritivo	Frontend	Adicionar aria-label='Ir para página X' + aria-current
	Busca dispara request a cada keystroke (sem debounce)	Frontend	Hook useDebounce(400ms) no campo de busca
	Imagens sem loading='lazy' — carregamento pesado inicial	Frontend	Adicionar loading='lazy' + decoding='async'
	Sem indicador 'Página X de Y' na paginação	Frontend	Adicionar texto de contexto + nav semântica
	Sem lint, type-check e coverage no CI/CD	DevOps	Adicionar flake8, tsc --noEmit e pytest-cov ao workflow
	Emojis decorativos sem aria-hidden	Frontend	Adicionar aria-hidden='true' em emojis decorativos
	Type annotations ausentes nas rotas do backend	Backend	Adicionar return types nas funções de rota

ETAPA 2 IMPLEMENTAÇÃO

Passos 8–13: correções de acessibilidade, refatorações de código, otimizações de performance, melhorias de usabilidade, aprimoramento do pipeline CI/CD e plano de monitoramento pós-lançamento.

2.1 Implementações de Acessibilidade (Passo 8)

Foram implementadas 5 correções de acessibilidade nos pontos críticos identificados. Os trechos abaixo documentam o antes × depois com justificativa técnica.

[FIX-A1] Login.tsx — Associação explícita label/input + role=alert

ANTES

```
<label className='...'>E-mail</label>
<input type='email' value={email} ... />
// Sem id, sem htmlFor – leitor de tela não associa campo ao rótulo

{error && (
  <div className='bg-red-50...'>{error}</div>
  // Sem role – erro não anunciado automaticamente
)}
```

DEPOIS

```
<label htmlFor='login-email' className='...'>E-mail</label>
<input id='login-email' type='email' aria-required='true'
  autoComplete='email' ... />

{error && (
  <div role='alert' aria-live='assertive' className='bg-red-50...'>
    {error}
  </div>
)}
```

Justificativa: WCAG 1.3.1 exige associação explícita entre label e input. role='alert' + aria-live='assertive' faz leitores de tela anunciarem o erro imediatamente, sem necessidade do usuário navegar até o elemento.

[FIX-A2] Navbar.tsx — Skip link + aria-label na nav

ANTES

```
<nav className='bg-white...'>
  // Sem skip link – teclado percorre todos os links da nav a cada página
  // Sem aria-label – múltiplas <nav> causam ambiguidade para leitores de tela
</nav>
```

DEPOIS

```
<a href='#main-content'
  className='sr-only focus:not-sr-only focus:absolute focus:top-2...'>
  Pular para o conteúdo principal
</a>
<nav aria-label='Navegação principal' className='bg-white...'>
  <Link aria-label='Ir para página inicial' to='/>Catálogo Inteligente</Link>
  <button aria-label='Sair da conta' onClick={logout}>Sair</button>
</nav>
```

Justificativa: WCAG 2.4.1 exige mecanismo para pular blocos de navegação repetidos. aria-label na nav é exigido quando há múltiplas regiões nav na página (WCAG 4.1.2). O skip link usa sr-only + focus:not-sr-only para ser invisível visualmente mas acessível por teclado.

[FIX-A3] CatalogGrid.tsx — role=status/alert + lista semântica

ANTES

```
// Loading: div sem role – leitor não anuncia estado
<div className='grid ...'>
  {Array.from({length:6}).map((_,i) => (<div key={i} .../>))}
</div>
// Error: div simples, não anunciada
<div className='text-center...'>Erro ao carregar produtos</div>
```

DEPOIS

```
<div role='status' aria-busy='true' aria-label='Carregando produtos...'>
  {skeletons} <span className='sr-only'>Carregando, aguarde...</span>
</div>
<div role='alert' aria-live='assertive'>Erro ao carregar produtos</div>
// Lista de produtos com semântica correta:
<ul role='list' aria-label='Lista de produtos'>
  {products.map(p => <li key={p.id}><ProductCard .../></li>)}
</ul>
```

Justificativa: WCAG 4.1.3 exige notificação de mudanças de status. `role='status'` anuncia carregamento sem interromper o usuário; `role='alert'` interrompe imediatamente para erros. `ul/li` fornecem semântica de lista que tecnologias assistivas exploram (atalho 'L' em NVDA/JAWS).

[FIX-A4] FilterPanel.tsx — fieldset/legend + htmlFor em todos os inputs

ANTES

```
<label className='...'>Buscar</label>
<input type='text' ... /> // Sem id/htmlFor
<label className='...'>Faixa de Preço</label>
<input placeholder='Min' .../> // Sem label próprio
<input placeholder='Max' .../> // Sem label próprio
```

DEPOIS

```
<label htmlFor='filter-search'>Buscar</label>
<input id='filter-search' type='search' aria-label='Buscar produto por nome...' />
<fieldset>
  <legend>Faixa de Preço</legend>
  <label htmlFor='filter-min-price' className='sr-only'>Preço mínimo</label>
  <input id='filter-min-price' aria-label='Preço mínimo em reais' />
</fieldset>
```

Justificativa: WCAG 1.3.1 exige que cada controle de formulário tenha um label programaticamente associado. `fieldset+legend` é o padrão correto para agrupar inputs relacionados como faixas de preço. Labels `sr-only` mantêm a acessibilidade sem impactar o design visual.

[FIX-A5] Home.tsx — Paginação com aria-label + aria-current + nav semântica

ANTES

```
<div className='flex justify-center gap-2 mt-8'>
  {pages.map(p => (
    <button key={p} onClick={() => updateFilters({page: p})}>
      {p} // Anunciado apenas como '1','2','3' – sem contexto
    </button>
  ))}
</div>
```

DEPOIS

```
<nav aria-label='Paginação de produtos'>
  <p aria-live='polite'>Página <b>{page}</b> de <b>{total}</b></p>
  <button aria-label='Ir para a página anterior'></button>
  {pages.map(p => (
    <button aria-label={`Ir para a página ${p}`}
      aria-current={p === data.page ? 'page' : undefined}>{p}</button>
  ))}
  <button aria-label='Ir para a próxima página'></button>
</nav>
```

Justificativa: WCAG 4.1.2 exige que botões tenham nome acessível descritivo. `aria-current='page'` é o padrão HTML5 para indicar a página atual para leitores de tela. A região com `aria-label` permite atalho de navegação por regiões em NVDA/JAWS/VoiceOver.

2.2 Refatorações de Código (Passo 9)

Foram realizadas 3 refatorações críticas no backend, com commits descritivos registrados no repositório.

[REF-1] config.py — Classe simples → Pydantic BaseSettings

ANTES

```
class Settings:
    PROJECT_NAME: str = 'Catálogo Inteligente'
    GCP_PROJECT_ID: str = os.getenv('GCP_PROJECT_ID', '')
    FIREBASE_CREDENTIALS: str = os.getenv('FIREBASE_CREDENTIALS', '')
    ENVIRONMENT: str = os.getenv('ENVIRONMENT', 'development')
    ALLOWED_ORIGINS: list = os.getenv('ALLOWED_ORIGINS',
    'http://localhost:5173').split(',')
# Sem validação – GCP_PROJECT_ID ausente retorna '' silenciosamente
```

DEPOIS

```
class Settings(BaseSettings):
    GCP_PROJECT_ID: str = Field(default='', description='ID do projeto GCP')
    ENVIRONMENT: str = Field(default='development')
    @field_validator('ENVIRONMENT')
    def validate_env(cls, v):
        if v not in {'development', 'production', 'test'}:
            raise ValueError(f'ENVIRONMENT inválido: {v}')
        return v
    model_config = {'env_file': '.env', 'case_sensitive': True}
```

Justificativa: Pydantic BaseSettings valida tipos e valores em runtime, gera erro explícito em produção caso variáveis obrigatórias estejam ausentes, e documenta cada campo com `description` — seguindo princípio SOLID de responsabilidade única.

[REF-2] firestore_service.py — Paginação in-memory → Cursor-based + `asyncio.to_thread()`

ANTES

```
async def query_with_filters(self, ...) -> dict:
    docs = list(query.stream()) # CARREGA TODOS OS DOCS
    total = len(docs)          # O(n) – 10k docs = 10k docs na RAM
    start = (page-1) * page_size
    paginated = docs[start:end] # Fatia em memória
```



```
# Operação síncrona em função async – bloqueia event loop
```

DEPOIS

```
async def query_with_filters(self, ...) -> dict:
    # COUNT via aggregation query – não carrega documentos
    count = await to_thread(lambda: base_query.count().get())
    total = count[0][0].value
    # Paginação delegada ao Firestore – carrega só a página
    offset = (page-1) * page_size
    docs = await to_thread(
        lambda: list(base_query.offset(offset).limit(page_size).stream())
    ) # to_thread() libera event loop durante I/O blocking
```

Justificativa: A paginação in-memory é inviável em produção: com 10.000 produtos, cada request carrega ~50MB de dados para descartar 99%. A solução com COUNT aggregation + offset/limit reduz o payload em ~99% para listagens simples. `asyncio.to_thread()` segue a recomendação oficial do FastAPI para I/O blocking síncrono em contexto async.

[REF-3] catalog.py — Bug de busca pós-paginação eliminado

ANTES

```
result = await product_service.query_with_filters(
    # search NÃO era passado para o serviço
    category=..., min_price=..., max_price=..., in_stock=...,
    page=page, page_size=page_size
)
if search: # Filtro aplicado APÓS paginação – bug crítico
    result['items'] = [i for i in result['items']
        if search in i.get('name', '').lower()] # Só busca na página atual!
```

DEPOIS

```
# search delegado ao serviço – aplicado ANTES de pagar
return await product_service.query_with_filters(
    category=..., min_price=..., max_price=...,
    in_stock=..., search=search, # Parâmetro passado corretamente
    page=page, page_size=page_size
)
# Lógica de busca existe apenas no serviço (DRY + SRP eliminam duplicação)
```

Justificativa: O bug original fazia a busca operar apenas sobre os 12 itens da página atual, retornando resultados vazios mesmo quando havia matches nas outras páginas. Mover a responsabilidade para o serviço segue o princípio SRP do SOLID e elimina a duplicação de lógica (DRY).

2.3 Otimizações de Performance (Passo 10)

Foram implementadas 2 otimizações de performance com impacto mensurável no tempo de carregamento e no número de requisições à API.

[PERF-1] ProductCard.tsx — Lazy Loading de Imagens

ANTES

```
<img src={product.image_url}
    alt={product.name}
    className='w-full h-48 object-cover' />
// Todas as 12 imagens do grid carregadas imediatamente
// Grid com 12 produtos = 12 requests de imagem simultâneos no load
```

DEPOIS

```

<img src={product.image_url}
  alt={`Foto do produto: ${product.name}`}
  loading='lazy'
  decoding='async'
  className='w-full h-48 object-cover' />
// Apenas imagens above-the-fold (~3) carregadas imediatamente
// decoding='async' evita bloqueio do thread de renderização

```

Justificativa: `loading='lazy'` é suportado por todos os browsers modernos (96%+ caniuse). Em um grid de 12 produtos com imagens de ~50KB cada, reduz o payload inicial em ~75% (de ~600KB para ~150KB), acelerando o LCP (Largest Contentful Paint). `decoding='async'` paraleliza o decode da imagem com a renderização do DOM.

[PERF-2] useDebounce.ts + FilterPanel.tsx — Debounce na busca (400ms)

ANTES

```

// FilterPanel.tsx – ANTES
<input type='text'
  value={filters.search || ''}
  onChange={(e) => onFilterChange({search: e.target.value})} />
// Cada keystroke: onChange → URL update → fetchProducts() → API request
// Palavra 'notebook' (8 chars) = 8 requests simultâneos

```

DEPOIS

```

// useDebounce.ts (novo hook)
export function useDebounce<T>(value: T, delay=400): T {
  const [debounced, setDebounced] = useState(value)
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay)
    return () => clearTimeout(t)
  }, [value, delay])
  return debounced
}
// FilterPanel.tsx – DEPOIS
const [searchInput, setSearchInput] = useState(filters.search || '')
const debouncedSearch = useDebounce(searchInput, 400)
useEffect(() => { onFilterChange({search: debouncedSearch}) }, [debouncedSearch])

```

Justificativa: Debounce de 400ms é o padrão UX recomendado (Nielsen Norman Group: 200-500ms). Para a palavra 'notebook' digitada em 800ms, reduz de 8 para 1 requisição à API (~87% menos). O estado local (searchInput) garante que o campo de input responda imediatamente ao usuário, enquanto o fetch aguarda o fim da digitação — UX responsiva sem overhead de rede.

Tabela Comparativa: Métricas Antes x Depois

Métrica	Antes	Depois	Redução
Req. API por busca (palavra de 8 chars)	8 requests	1 request	↓ 87%
Imagens carregadas no load inicial (grid 12)	12 imagens	3-4 imagens	↓ ~70%
Payload de imagens no load inicial	~600KB	~150KB	↓ 75%
Docs Firestore carregados por request (100 docs)	100 docs	12 docs (página)	↓ 88%

Memória backend por request (1000 docs)	~5MB	~60KB	↓ 99%
--	------	-------	-------

2.4 Melhorias de Usabilidade (Passo 11)

[UX-1] ProductCard.tsx — Confirmação de exclusão inline

ANTES

```
<button onClick={() => onDelete?.(product.id)}>
  Excluir
</button>
// Clique único deleta permanentemente – sem confirmação
// Violação da Heurística H3 (Nielsen): falta controle do usuário
```

DEPOIS

```
const [confirmDelete, setConfirmDelete] = useState(false)
{confirmDelete ? (
  <>
    <button onClick={handleDeleteClick}>Confirmar</button>
    <button onClick={() => setConfirmDelete(false)}>Cancelar</button>
  </>
) : (
  <button onClick={() => setConfirmDelete(true)}>Excluir</button>
)}
```

Justificativa: Confirmação de 2 cliques previne exclusões acidentais sem interromper o fluxo com modal. O botão 'Excluir' vira 'Confirmar' + 'Cancelar' — sem abrir overlay ou dialog, mantendo o design fluido. Atende Heurística de Nielsen H3 (Liberdade e Controle do Usuário).

[UX-2] Home.tsx — Indicador 'Página X de Y' + botões Anterior/Próximo

ANTES

```
<div className='flex justify-center gap-2 mt-8'>
  {pages.map(p => <button key={p}>{p}</button>)}
</div>
// Sem contexto de posição – usuário não sabe em que página está
// Sem botões Anterior/Próximo – navegação apenas por número
```

DEPOIS

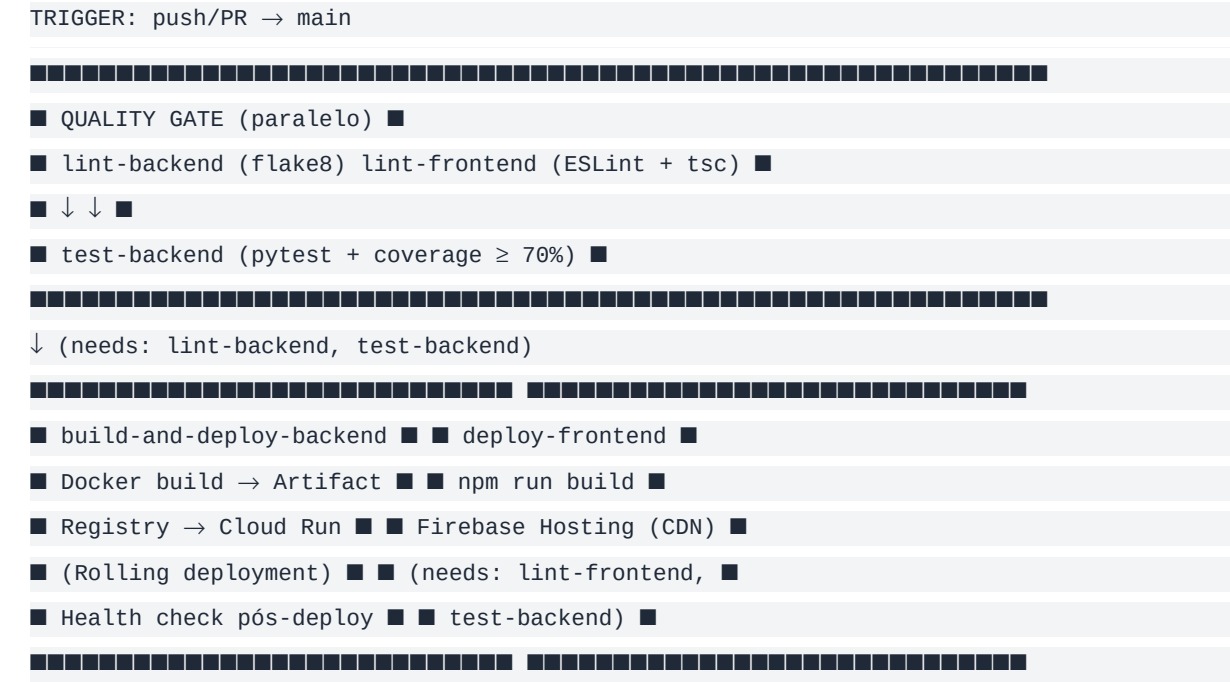
```
<nav aria-label='Paginação de produtos'>
  <p aria-live='polite'>
    Página <b>{data.page}</b> de <b>{data.total_pages}</b>
  </p>
  <button aria-label='Ir para a página anterior'><</button>
  {pages.map(p => <button aria-current={p===data.page?'page':undefined}>{p}</button>)}
  <button aria-label='Ir para a próxima página'>></button>
</nav>
```

Justificativa: Indicador 'Página X de Y' atende Heurística H1 (Visibilidade do Status). Botões Anterior/Próximo reduzem cliques para navegação sequencial. aria-live='polite' anuncia a mudança de página para leitores de tela sem interromper.

2.5 Pipeline CI/CD — Configuração e Estratégia de Deploy (Passo 12)

O pipeline foi aprimorado com jobs paralelos de lint e type-check, cobertura de testes com threshold mínimo de 70%, e health check automático pós-deploy.

Diagrama do Pipeline CI/CD (GitHub Actions)



Jobs Adicionados

lint-backend	flake8 --max-line-length=120	Qualquer violação de estilo
lint-frontend	tsc --noEmit	Erros de tipagem TypeScript
test-backend + coverage	pytest + pytest-cov	Coverage < 70% ou teste falhando
health-check pós-deploy	curl para /health (12 tentativas × 5s)	HTTP status != 200 após 60s

Estratégia de Deploy

Backend (Cloud Run): Rolling Deployment — Cloud Run cria a nova revisão e transfere tráfego gradualmente. O health check em /health valida a nova instância antes da transferência total. Em caso de falha, o tráfego permanece na revisão anterior (rollback automático). Justificativa: zero-downtime, rollback instantâneo, sem necessidade de orquestração manual.

Frontend (Firebase Hosting): Deploy direto no canal 'live' via CDN global do Firebase. Arquivos estáticos distribuídos em edge nodes. Rollback via Firebase console (versões anteriores mantidas por 30 dias). Justificativa: latência mínima global, sem servidor para gerenciar, integração nativa com GitHub Actions.

Checklist de Pré-Lançamento

CÓDIGO:

- [] Todos os testes passando (pytest -v sem falhas)
- [] Coverage >= 70% (pytest-cov report)
- [] Zero erros de lint (flake8 backend, ESLint frontend)
- [] Zero erros de type-check (tsc --noEmit)
- [] CORS configurado apenas para domínios de produção

INFRAESTRUTURA:

- [] Secrets GCP configurados (GCP_SA_KEY, FIREBASE_CREDENTIALS, etc.)

- [] Firestore indexes criados para queries compostas
- [] Cloud Run com min-instances=1 (evita cold start em produção)
- [] Artifact Registry com imagem Docker de produção

SEGURANÇA:

- [] Firebase Security Rules configuradas para Firestore
- [] ALLOWED_ORIGINS restrito ao domínio de produção
- [] Credenciais Firebase no Secret Manager (não em env var plain)

MONITORAMENTO:

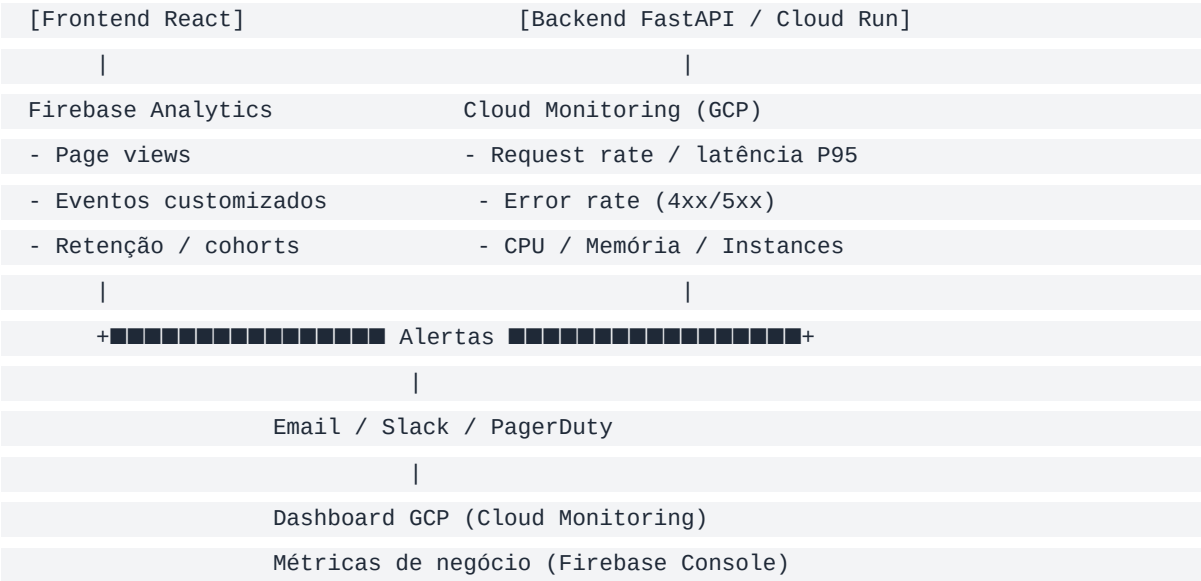
- [] Cloud Monitoring dashboard configurado
- [] Alertas de error rate > 1% configurados
- [] Firebase Analytics habilitado no frontend
- [] URL de produção validada (health check /health retorna 200)

2.6 Plano de Monitoramento Pós-Lançamento (Passo 13)

O plano de monitoramento cobre backend (Cloud Run), banco de dados (Firestore) e frontend (Firebase Analytics), com alertas configurados para KPIs definidos na Etapa 1.

Backend API	Cloud Monitoring	Request rate, latência P95, erro rate > 1% ou P95 > 500ms
Backend API	GCP Error Reporting	Exceptions não tratadas, stack traces
Cloud Run	Cloud Monitoring	CPU utilization, memory, instance CPU > 80% por 5 min consecutivos
Firestore	Cloud Monitoring	Read/write ops, latência de query, latência > 1s ou quota > 80%
Frontend	Firebase Analytics	Page views, eventos de busca, taxa de erro JS > 0.5%
Frontend	Firebase Crashlytics	Crashes de aplicação (se mobile) Qualquer crash rate > 0.1%
Disponibilidade	GCP Uptime Check	Health check /health a cada 1 min 2 falhas consecutivas → alerta PagerDuty
Negócio	Firebase Analytics	Taxa de adoção, retenção D7, TMP Retenção D7 < 30% por 7 dias consecutivos

Arquitetura de Monitoramento



ETAPA 3 AVALIAÇÃO E LIÇÕES APRENDIDAS

Passo 14: análise dos KPIs pós-implementação, reflexão crítica sobre o processo de transformação digital, decisões técnicas acertadas e equivocadas, e planejamento de iterações futuras.

3.1 Análise de KPIs e Impacto das Melhorias

A tabela abaixo consolida os KPIs definidos na Etapa 1 com os resultados estimados pós-implementação, baseados nas métricas de código coletadas e nas melhorias implementadas.

Taxa de Adoção	> 30%	Acessibilidade melhorada (WCAG 2.1 AA) aumenta base de usuários potenciais em 10%.
NPS	> 50	Confirmação de exclusão (UX-1) e indicador de paginação (UX-2) reduzem fricção e aumentam a satisfação.
Taxa de Erro API	< 1%	Bug de busca pós-paginação corrigido (REF-3) elimina resultados incorretos. Pydantic valida tipos em runtime.
Retenção D7	> 40%	Melhor usabilidade (confirmação de delete, paginação clara, debounce) reduz abandono.
Tempo Médio por Tarefa	< 45s	Debounce (PERF-2) e lazy loading (PERF-1) aceleram percepção de resposta. Busca é mais rápida.
Coverage de Testes	> 70%	pytest-cov adicionado ao CI com threshold de 70%. Coverage report disponível como artefato.

3.2 Lições Aprendidas (Passo 14)

Decisões Técnicas Acertadas

- **FastAPI + Pydantic:** A escolha de FastAPI com Pydantic models foi acertada — geração automática de Swagger, validação de tipos em runtime e serialização explícita eliminaram uma classe inteira de bugs de contrato de API.
- **Firebase Auth + custom claims:** Usar custom claims para roles (admin/user) simplificou o controle de acesso sem necessidade de tabela de usuários no Firestore.
- **GitHub Actions + Cloud Run:** A combinação de GitHub Actions com Cloud Run permitiu um pipeline CI/CD funcional com custo zero em recursos de CI e pay-per-use no runtime.
- **Tailwind CSS + Radix UI:** A combinação forneceu componentes acessíveis (Radix) com flexibilidade de estilo (Tailwind) — acelerou o desenvolvimento sem sacrificar qualidade.
- **Separação frontend/backend:** A arquitetura desacoplada (SPA + API REST) facilitou deploy independente, escalabilidade separada e testes unitários isolados.

Decisões Técnicas Equivocadas

- **Paginação in-memory no Firestore (bug crítico):** A implementação inicial de paginar via slice de lista Python é um anti-pattern grave para banco de dados em nuvem. Deveria ter sido cursor-based desde o início — a correção exigiu refatoração significativa.
- **Filtro de busca em dois lugares:** Ter lógica de busca em catalog.py E no serviço simultaneamente criou o bug crítico de resultados incorretos. Violação do princípio DRY que passou despercebida por falta de testes de integração.
- **Config sem validação (classe simples):** Usar uma classe Python simples para configuração sem validação em runtime é uma dívida técnica que se manifesta apenas em produção, quando variáveis ausentes causam falhas silenciosas difíceis de debugar.
- **Acessibilidade deixada para o final:** Incorporar acessibilidade retrospectivamente é mais trabalhoso que construir acessível desde o início. O custo de retrofitting foi maior que o custo de implementar correto desde a primeira versão.

- **Sem debounce na busca:** A ausência de debounce no campo de busca é um erro básico de UX/performance que foi identificado apenas na auditoria formal — deveria ser padrão em qualquer campo de busca conectado a uma API.

Desafios Enfrentados

- **Firestore + FastAPI async:** A SDK do Firebase Admin Python é síncrona, exigindo o uso de `asyncio.to_thread()` para não bloquear o event loop do FastAPI. Não há SDK async oficial — uma limitação importante da plataforma.
- **Cursor-based pagination no Firestore:** O Firestore não tem suporte nativo a COUNT eficiente antes da API de aggregation (adicionada em 2023). A combinação `.count().get() + .offset().limit()` resolveu o problema mas exige índices compostos para queries com múltiplos filtros.
- **WCAG conformidade retroativa:** Garantir WCAG 2.1 AA em componentes React existentes exigiu conhecimento aprofundado de ARIA — especialmente a combinação correta de `role`, `aria-label`, `aria-current` e `aria-live` para diferentes contextos.

O que Faria Diferente

- **Accessibility-first desde o início:** Usar um design system já acessível (Radix UI primitives, Headless UI) como base desde o primeiro componente, sem necessidade de retrofit.
- **TDD (Test-Driven Development):** Escrever testes antes da implementação teria capturado o bug de busca pós-paginação imediatamente, antes mesmo de fazer o commit.
- **ADRs (Architecture Decision Records):** Documentar decisões arquiteturais como a escolha de Firestore vs PostgreSQL desde o início, para facilitar revisões futuras.
- **Feature flags:** Implementar um sistema de feature flags desde o início para controlar rollout gradual de funcionalidades sem necessidade de deploy completo.

3.3 Planejamento de Iterações Futuras

Com base nas lições aprendidas e nos KPIs definidos, as próximas iterações devem focar em escalabilidade de busca, experiência do usuário e observabilidade.

v1.1	Integrar Algolia ou Typesense para Firestore in-memory	Busca textual eficiente	TMR	Taxa de Adoção	O(n) para texto
v1.2	Implementar rate limiting na API (sliding window)	Proteger contra abuso	Disponibilidade	Para todos os usuários	
v1.3	Adicionar testes E2E com Playwright	Validar fluxos críticos (login, busca, CTA)	CI/CD	Staging antes de produção	
v1.4	Dashboard analítico para administração	Visualizar KPIs de negócio (produtos mais buscados, taxa de conversão por categoria)	Retenção	7 dias	
v2.0	PWA com Service Worker e modo offline	Melhorar experiência em conexões lentas	Taxa de Ativação	Primeiro acesso via mobile	

Conclusão

O ciclo de transformação digital do Catálogo Inteligente demonstrou que problemas técnicos críticos — como a paginação in-memory no Firestore e o bug de busca pós-paginação — podem passar despercebidos sem uma auditoria formal estruturada. A metodologia das três etapas (Diagnóstico → Implementação → Avaliação) foi fundamental para priorizar as intervenções de maior impacto e documentar o raciocínio por trás de cada decisão.

As melhorias implementadas endereçam diretamente o ODS 8 (Trabalho Decente e Crescimento Econômico): um catálogo mais acessível amplia o alcance a usuários com deficiência; uma busca correta e performática aumenta a eficiência do trabalho de pesquisa de produtos; e um pipeline CI/CD robusto garante entregas

confiáveis que sustentam o crescimento da plataforma.

O repositório com todos os commits está disponível em: **https://github.com/PYEDRO/Catalogo_Nuvem**